

Falling Sand Game : Sandbox

SIMON CHARBONNEAU, SACHA GREGOIRE, HAI AN MAI

6 June 2026

§1 Team Members and Responsibilities

- Simon Charbonneau:
 - Setup for the sequential version : `godot-cpp`.
 - Displaying the grid, UI keys, and buttons in godot in both versions.
 - Implementing texturing of particles.
 - Implementing the directional phase implementation on the GPU version.
- Sacha Gregoire:
 - Improving the sequential code: adding randomization for the order of cell updates, adding `updated` array to fix interaction errors.
 - Implementing the threaded version in the CPU from the sequential code.
 - Implementing the Margolus neighborhoods in the GPU version.
- Hai An Mai:
 - Implementing the first logic for the sequential implementation, including interactions between `Sand`, `Rock`, `Water` and `Vapor`.
 - Setup for GPU version: adapting the `GDScript` and `GLSL` from the GOL implementation.
 - Adding a feature to save presets for benchmarking in the GPU version.
 - Creating and running benchmarks for all versions.

§2 Introduction

Falling Sand Game is a simulation of simple physics of falling sand and other particles. In this report we will go through our different attempts, choices, and implementations done during this project.

As mentioned in the project description, our simulation is based on a cellular automaton; a grid of cells evolving through time depending on the neighboring cells. The main structure is a large grid of pixels, where each pixel will correspond to a specific particle.

We started the simplest implementation on the CPU using only `Air` and `Sand`. After we figured out the logic, we added interactions with `Water` and `Vapor`. Finally, the last thing we have done on the CPU side is implement CPU threading from the sequential code.

Afterwards we moved on to implementing the GPU version. We had to change some implementation logic as to avoid race conditions while computing all cells in parallel. Lastly, we have implemented Margolus neighborhoods, as the logic is much easier to understand than the first version on the GPU. Moreover, we wanted to see the performance difference between the two algorithms.

To be able to test our implementations and to create benchmarks, we added user input to be able to spawn new particles in the grid, count particles of each type to check particle preservation, and save and load presets.

§3 Repositories

Our project consists of two parts: **sequential** and **GPU**. We have decided to make two repositories for the two parts to account for the difference in the code structure.

Sequential: <https://gitlab.binets.fr/hai-an.mai/sandbox>

GPU: https://gitlab.binets.fr/hai-an.mai/sandbox-gpu/-/tree/final_version

§4 Sequential implementation

The goal of this part was to get the feel of the simulation and logic of falling sand.

§4.1 Setup

For the sequential code we made a `godot-cpp` template to make the first implementation of the simulation on CPU be done in C++.

We have introduced a `Grid` class to represent the grid:

```
class Grid {
private:
    size_t width, height;
    std::vector<Mat> cells;
```

representing a rectangle with `width` and `height`, each cell contains the enum class `Mat` representing different type of material: `Air`, `Sand`, `Water`, `Stone`, `Vapor`.

Each `Mat` has a one-to-one mapping to a `Particle`, which contains all their properties:

```
struct Particle {
    Mat m;
    std::string name;
    State state = State::Static;
    uint8_t density = 0;
    uint8_t dispersion_rate = 0;
};
```

Notably we have `State` storing possible states: `Static`, `Solid`, `Liquid`, `Gas`.

We used `density` to determine which particles we should swap. The idea is that if a particle with higher density is on top of a particle with lower density, we would swap the location of the two particles (this will be explained more in detail later). We for example set `Air` to have density `0` and `Sand` to have density `100`.

§4.2 Logic

The sequential code will update all cells sequentially going through the whole grid.

```
void Grid::update_seq() {
    bool ttob = coin();           // whether or not we go top to bottom
    for(size_t y = ttob ? 0 : height; ttob ? y < height : y-- > 0; y += ttob) {
        bool ltor = coin();       // whether or not we go left to right on this row
        for(size_t x = ltor ? 0 : width; ltor ? x < width : x-- > 0; x += ltor) {
            update_cell(x, y);
        }
    }
}
```

We added randomization (using `coin()` which is just a fair coin) on the order of updating up-down and left-right, to make the simulation more realistic and less deterministic. We are also keeping the array `updated` if the cell was already updated within the current update process.

The `update_cell(x,y)` is implemented with the following logic:

- If the particle is `Air` or has a state `Static`: we do nothing
`if (cur.m == Mat::Air || cur.state == State::Static) return;`
- If the particle has a higher density than the cell below: we swap the two particles.

```
// Vertical
int dir_ver = (cur.state == State::Gas) ? -1 : 1;
bool vert = check_valid(x, y, x, y + dir_ver);
if(vert) { swap(x, y, x, y + dir_ver); return; }
```

- If the particle did not manage to move, we check the lower diagonals:
 - If both diagonals are available: choose one at random to swap
 - If only one is available: swap with it

```
// Diagonal
bool left = check_valid(x, y, x - 1, y);
bool right = check_valid(x, y, x + 1, y);
bool diag_left = left && check_valid(x, y, x - 1, y + dir_ver);
bool diag_right = right && check_valid(x, y, x + 1, y + dir_ver);
if(diag_left && diag_right) {
    if (coin()) swap(x, y, x - 1, y + dir_ver);
    else swap(x, y, x + 1, y + dir_ver);
}
else if(diag_left) swap(x, y, x - 1, y + dir_ver);
else if(diag_right) swap(x, y, x + 1, y + dir_ver);
```

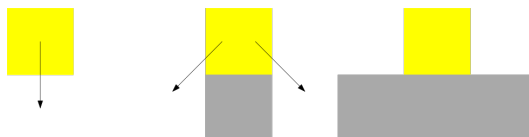


Figure 1: Basic rules of sand

- If the particle still did not manage to move and is a `Liquid`, we check if the sides are available:

- If left and right are available: choose one at random
- If only one is available: swap with it

```

if(diag_left || diag_right || cur.state == State::Solid || (!left && !right))
    return;
if (left && right) {
    if (coin()) left = false;
    else right = false;
}
if (left) {
    size_t i = check_valid_iter(x, y, x - 1, y, cur.dispersion_rate, true);
    swap(x, y, x - i, y);
}
if (right) {
    size_t i = check_valid_iter(x, y, x + 1, y, cur.dispersion_rate, true);
    swap(x, y, x + i, y);
}

```

We use the `dispersion_rate`, a quantity that allows a particle to travel more than 1 pixel per frame.

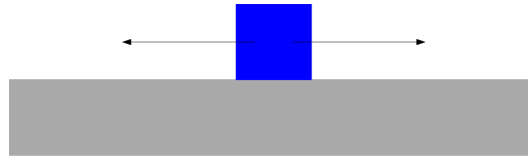


Figure 2: Basic rules of water

- If the particle has a state `Gas`, we do the same procedure as for water, but check the upper diagonals instead of lower diagonals

```

int dir_ver = (cur.state == State::Gas) ? -1 : 1;

```

We wished to add more features to this sequential code. However, due to time constraints, we stopped here. As a final nudge for this section, we have implemented a concurrent version of this simulation using CPU threads. We divided the grid into columns of width `stripe_w`. We first update all of the odd columns, and then update all of the even columns. To avoid weird movements on cells in between two columns, we apply an offset to all columns every time we update the grid.

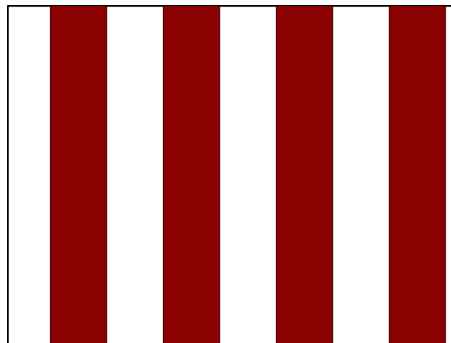


Figure 3: Threaded sequential code example columns

§5 Concurrent - GPU

§5.1 GPU: GDScript and GLSL

For a GPU implementation, we had to switch to `GDScript` and `GLSL`. We used the GOL template given to us. On the `GDScript` side, the most important part is the `_process` function:

```
func _process(_delta: float) -> void:
    if not rd or not pipeline.is_valid():
        return
    for i in range(6):
        total_frames += 1
        rd.buffer_clear(counter_buffer, 0, 16)
        rd.texture_copy(
            textures[1],
            textures[0],
            Vector3(0,0,0),
            Vector3(0,0,0),
            Vector3(WIDTH, HEIGHT, 1),
            0, 0, 0, 0
        )

    var compute_list = rd.compute_list_begin()
    rd.compute_list_bind_compute_pipeline(compute_list, pipeline)
    rd.compute_list_bind_uniform_set(compute_list, uniform_set, 0)

    var push_bytes = make_push_bytes()
    rd.compute_list_set_push_constant(compute_list, push_bytes, push_bytes.size())
    # -----

    var groups_x = (WIDTH + 15) / 16
    var groups_y = (HEIGHT + 15) / 16
    rd.compute_list_dispatch(compute_list, groups_x, groups_y, 1)
    rd.compute_list_end()
```

Instead of rotating between two texture grids to set the compute pipeline, we copy the output grid into the input grid at the beginning of each time step. That way, we only have to draw particles that have moved instead of drawing every particle. We additionally push extra bytes that are used to implement the user input.

We decided to stay with the 16×16 group size. This is a reasonable compromise for several reasons. It maps naturally to a square tiling of the grid, it provides 256 invocations per workgroup, and it remains comfortably below the common hardware limit of 1024 threads per workgroup. A smaller workgroup would increase scheduling overhead and reduce the amount of useful work per dispatch unit, while a much larger workgroup would risk reducing occupancy and placing unnecessary pressure on hardware resources. The 16×16 choice therefore balances parallelism, portability, and resource efficiency.

As for the shaders, we have copied the structure of particles from the sequential code. We have two grids including the colors of each cell: `input_grid` and `output_grid`, which contain the colors of each cell as a `vec4`. We thus needed a way to go between `vec4` and `mat` variables. To this end, we have `material_colors`, an array containing the colors of each material, as well as the `decode` function:

```

Particle decode(vec4 pixel) {
    for(int i = 0; i < count_material; i++) {
        if(length(pixel - material_colors[i]) < 0.15) {
            return properties[i];
        }
    }
    return properties[0];
}

```

§5.2 Directional phase implementation

In our directional phase GPU implementation, the logic for the particle simulation is handled by the `update` function. Because the shaders evaluate cells concurrently, we cannot just let particles move in any arbitrary directions without some care, as this would lead to race conditions. Therefore, we orchestrate the movement of all particles using a deterministic and alternating pattern. We give priority to particles moving vertically, and we restrict the direction of non-vertical movement in a given frame by calculating a 6-periodic phase based on the frame count.

```

void update(ivec2 coord) {
    int mat = get_mat(coord);
    if (getting_moved_into_vertically(coord) || drawing(coord)) return;
    Particle cell = properties[mat];

    int phase = params.frame_count % 6;
    int move_x = (phase == 0 || phase == 1 || phase == 2) ? -1 : 1;
    int move_y = (phase == 0 || phase == 3) ? -1 : (phase == 1 || phase == 4) ? 0 : 1;
    // phase 0: up left
    // phase 1: flat left
    // phase 2: down left
    // phase 3: up right
    // phase 4: flat right
    // phase 5: down right

    switch (cell.state) {
        case Static: break;
        case Solid:
            update_solid(coord, move_x, move_y);
            break;
        case Liquid:
            update_liquid(coord, move_x, move_y);
            break;
        case Gas:
            if (cell.mat != Air) update_gas(coord, move_x, move_y);
            break;
    }
}

```

Note that, if the cell is getting drawn into or moved into vertically, we skip the computation phase entirely.

§5.2.1 Logic

Thanks to this cycle, combined with the fact that we render every sixth frame in `texture_rect.gd`, the particles seamlessly disperse and tumble in all directions over

time without competing for the same target pixels, and this helps avoid cell duplication or deletion errors. Once the phase direction is determined, we decode the current cell's color into a particle to evaluate its properties, and then delegate the update logic based on the state of the particle :

- **Static** : Skips computation (cannot move)
- **Solid** : Solids prioritise falling vertically down and writing the **Gas** or **Liquid** particle below them into the current cell. If they cannot move vertically, the particle attempts a diagonal fall based on the phase direction, succeeding with probability equal to its **topple_prob**.
- **Liquid** : Liquids prioritise writing the **Solid** particle above in the current cell. Otherwise, the particle tries to move vertically, writing the **Gas** particle below them in the current cell. If neither succeeds, the particle attempts to move in the phase direction (if valid), up to as far as their **dispersion_rate**.
- **Gas** : Gases prioritise writing the **Solid** or **Liquid** above them into the current cell. Otherwise, the particle moves in the phase direction with some probability based on its **topple_prob**, skewed to slightly prefer going upwards.
- **Air** : Skips computation, as updates will be handled by the particle moving into the cell.

For example, the **get_liquid_move** function determines how far a liquid can disperse horizontally or diagonally during a frame. Looping until the liquid's **dispersion_rate**, it iterates along the target direction vector (dx , dy) and verifies that the liquid may move there, based on the particles at or above that target cell.

```
int get_liquid_move(ivec2 source, int dx, int dy, int dispersion, int frame) {
    if (dy == -1) return 0;
    if (!coin(source, frame)) return 0;

    for (int i = 0; i < dispersion; i++) {
        ivec2 target = source + ivec2(dx * (i+1), dy * (i+1));

        // Do not disperse through non-air particles
        if (get_mat(target) != Air) return i;
        // Do not disperse if the cell above the target is moving into it
        if (getting_moved_into_vertically(target)) return i;
        // Do not disperse through cells being drawn in
        if (drawing(target)) return i;
    }
    return dispersion;
}
```

§5.2.2 Correctness

To check correctness, we added a counter for each type of particle and made sure that it stayed consistent.

To address race conditions, we have mentioned that we use directional phases to make sure that we avoid the situation of multiple particles moving to the same cell.

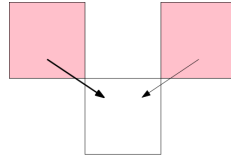


Figure 4: A case of a collision which we avoid by considering directional phases

§5.3 Margolus neighborhoods

As suggested by the project description, we also made a Margolus neighborhood implementation. The main advantage of this method is that its block-based structure removes race conditions by construction. Instead of evaluating cells individually, this implementation partitions the grid into 2×2 blocks. Because these blocks are disjoint, threads can safely update all four cells within a block simultaneously without any race conditions. To prevent particles from being trapped within a block, we shift the partitioning grid based on the current frame.

```
int offset = params.frame_count % 4;
int offset_x = (offset == 1 || offset == 2) ? 1 : 0;
int offset_y = (offset == 1 || offset == 3) ? 1 : 0;
```

§5.3.1 Logic

Except for some dedicated logic to handle the boundaries, the logic for the main grid is delegated to the representative of each neighborhood, the top-left cell of each 2×2 block. For each particle in the block, we rank them by density and check their relative rank using the encoding 0: is less dense, 1 is equally dense, 2 is more dense than the considering particle. We then generate a unique key, and use this key as an index in a lookup table `marg[81]`, which returns the appropriate swap operations. Some of these operations require a coin flip to incorporate material properties like `topple_prob`.

```
void update_marg(ivec2 coord) {
    ...
    // Return for cells that are not the representative (TL) of
    // their margolus neighborhood
    if (coord.x % 2 != offset_x || coord.y % 2 != offset_y) return;
    // Movement logic
    vec4 TL_color = imageLoad(input_grid, coord);
    vec4 TR_color = imageLoad(input_grid, coord + ivec2(1, 0));
    vec4 BL_color = imageLoad(input_grid, coord + ivec2(0, 1));
    vec4 BR_color = imageLoad(input_grid, coord + ivec2(1, 1));
    Particle TL = decode(TL_color);
    Particle TR = decode(TR_color);
    Particle BL = decode(BL_color);
    Particle BR = decode(BR_color);
    ivec4 key = ivec4(0, 0, 0, 0);
    bool cont = true;
    while (cont) {
        // Update key
        ...
        // Lookup movement pattern
        int op = lookup(key);
        // Apply movement pattern
        switch (op) {
            case ERROR:
```

```

    break;
case NONE: break;
case TLBR:
    if (!coin(coord, params.frame_count, TL.topple_prob)) break;
    // Swap TL and BR
    swap(TL_color, BR_color);
    swap(TL, BR);
    swap(key.x, key.w);
    // Stop checking
    cont = false;
    break;
case TLTR:
    ...
    ...
}
}
// Write changes to grid
imageStore(output_grid, coord, TL_color);
imageStore(output_grid, coord + ivec2(1, 0), TR_color);
imageStore(output_grid, coord + ivec2(0, 1), BL_color);
imageStore(output_grid, coord + ivec2(1, 1), BR_color);
}

```

Here is an example of set of Margolus neighborhoods between two particles, white with lower density than black. Using the same idea from the paper [1], we have the following interactions between e.g. sand and air.

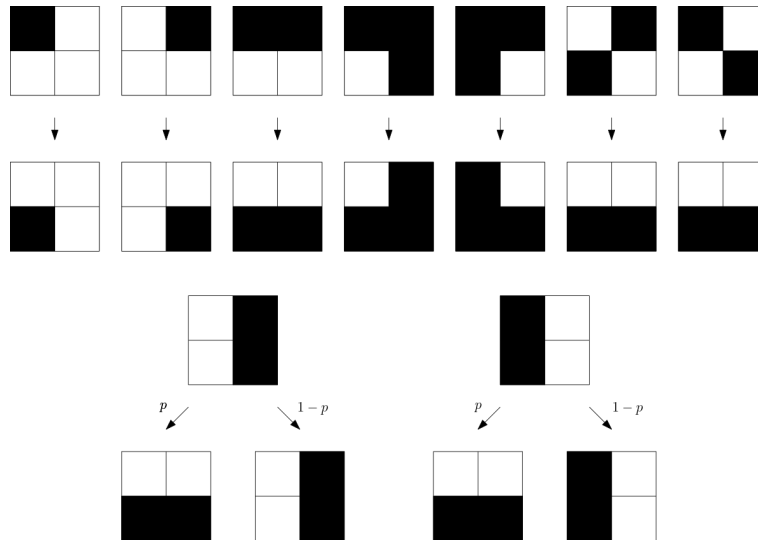


Figure 5: Margolus neighborhoods between Sand and Air

To account for **Liquid** movement, certain operations lead to sideways movement only if the considered particle is a liquid.

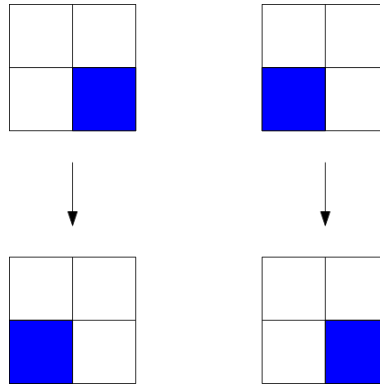


Figure 6: Additional Margolus neighborhoods for Water

§5.3.2 Correctness

The correctness of the Margolus neighborhoods is straightforward, as the 2×2 blocks do not interact with each other, so the only problems we would have is within a block. We encode every possible case, in such a way to ensure the particle preservation. Therefore, we conserve all the particles with every particle interaction.

§6 User Input

To make this game more interactive (and also for us to debug more easily), we implemented some UI features in the game. For the game on the GPU, these include

- **Mouse interaction** : the user can interact with the grid by using their mouse to spawn specific particles or erase.
- **Brush size** : the user can increase or decrease the size of the brush used to spawn or erase in the scene. This is done either by clicking the corresponding buttons in the scene or pressing the `ARROW_UP` and `ARROW_DOWN` keys.
- **Material selection** : the user can of course change which material they are spawning in the scene. This is done either by clicking the corresponding buttons in the scene or pressing the `S`, `W`, `R`, `V`, and `A` keys.
- **Clearing the grid** : the user can instantly clear the entire grid by setting all cells to air. This is done by pressing the `ESC` key.
- **Freezing the scene** : the user can freeze the particle update logic while keeping the renderer and user input active, allowing to for example carefully create complex scenes before playing them. This is done by pressing the `SPACE` bar.
- **Engine FPS** : the user can set the maximum FPS of the rendering engine, so as to slow down time and see more carefully the physics of the game. This is done by pressing the `+` and `-` keys.
- **Saving and loading presets** : the user can save scenes, and load saved scenes.

§7 Benchmarks

All tests were done on a laptop with

CPU: AMD Ryzen 7 7840HS w/ Radeon 780M Graphics

GPU: NVIDIA GeForce RTX 4050 Laptop GPU

In this section we will present some benchmarks and testing we have done on the models we have implemented.

§7.1 Sequential and CPU threads

In this section we focus on the following grid sizes:

128×128 , 256×256 , 512×512 , 1024×1024 , 2048×2048 , 4096×4096 .

We will first focus on the benchmarks on the sequential code. To test specific update functions, we used the following structure:

```
void update() {
    auto start = std::chrono::high_resolution_clock::now();
    update_type(); // <- the type of update we want to check
    auto end = std::chrono::high_resolution_clock::now();
    last_update_ms = std::chrono::duration_cast<std::chrono::microseconds>
        (end - start).count() / 1000.0;
};
double last_update_ms = 0.0;
```

We see the table of average runtime of each update function with corresponding number of threads with respect to the grid size. We notice due to an overhead from the initialization of threads, in the case of smaller grids, the `update_seq` is performing slightly better.

	update_seq	update_seq.threaded		
# threads	1	2	3	4
128×128	0.17	2.7	0.6	0.9
256×256	0.7	0.85	0.8	0.95
512×512	2.4	2.3	1.65	1.4
1024×1024	8.6	7.1	4.8	3.5
2048×2048	30.5	24.5	16	10.5
4096×4096	113	100	60	42.5

Table 1: Average runtime (in milliseconds) of update functions with CPU threads

# threads	$1 \rightarrow 2$	$1 \rightarrow 3$	$1 \rightarrow 4$
512×512	1.04	1.46	1.71
1024×1024	1.21	1.79	2.46
2048×2048	1.24	1.9	2.9
4096×4096	1.13	1.88	2.66

Table 2: The speed up from the sequential code to threaded code

In the larger cases starting from 512×512 we notice a speedup with the number of threads.

As expected, the sequential version performs poorly with larger grids. In this case, we would like to have more threads, as with a small amount of threads, the speed up is not very relevant.

We notice that the best speedup we get from the `update_seq` is using `update_seq_threaded` on 4 threads with the grid 2048×2048 .

§7.2 GPU

In this section we will test the two implementations on the GPU: **directional phases** and **Margolus neighborhood** on the following grid sizes:

$$1024 \times 1024, 1536 \times 1536, 2048 \times 2048, 3072 \times 3072, 4096 \times 4096, \\ 6144 \times 6144, 7168 \times 7168, 8192 \times 8192.$$

We will also test the models on given presets:

- `all-air` : all pixels are air
- `test-sand` : a big blob of sand
- `test-vapor` : a big blob of vapor
- `test-water` : a big blob of water

We now benchmark the directional phases implementation in GPU.

	Presets			
Resolution	<code>all-air</code>	<code>test-sand</code>	<code>test-vapor</code>	<code>test-water</code>
1024×1024	1.0043	1.0043	1.04	1.01
1536×1536	1.002	1.19	1.32	1.22
2048×2048	0.96	1.38	1.74	1.69
3072×3072	1.80	2.63	3.04	2.76
4096×4096	3.04	4.44	4.74	4.33
6144×6144	6.49	11.18	11.38	10.39
7168×7168	8.02	11.67	11.09	10.50
8192×8192	18.72	19.13	20.25	19.50

Table 3: Average runtime (in milliseconds) across different resolutions and material presets for the directional phase implementation

As expected, the GPU implementation is much faster than the CPU implementation. For the directional phase method, the runtime stays almost constant for small and medium grids, which suggests that the overhead of dispatching the compute shader dominates when the grid is still relatively small.

As the resolution increases, the runtime grows more clearly. In these cases, the `test-vapor` and `test-water` tests usually take the longest to run, as the logic for gas and liquid particles is slightly more complicated. In contrast, the `test-sand` test is slightly faster because the logic is slightly easier, and the `all-air` scene is faster because cells exit early from the update logic.

The relatively small performance difference between the `all-air` preset, as compared to the others, indicates that a substantial fraction of the runtime comes from overhead (such as launching a thread for every cell, performing memory accesses, etc.), even at higher resolutions.

Resolution	Presets			
	<code>all-air</code>	<code>test-sand</code>	<code>test-vapor</code>	<code>test-water</code>
1024×1024	1.06	1.02	1.03	1.07
1536×1536	1.07	1.17	1.17	1.19
2048×2048	1.42	1.64	1.64	1.15
3072×3072	1.44	2.06	2.36	2.21
4096×4096	2.72	4.06	4.36	3.99
6144×6144	5.92	8.72	9.46	8.58
7168×7168	8.01	11.69	12.70	11.84
8192×8192	18.72	19.13	20.23	19.07

Table 4: Average runtime (in milliseconds) for the different resolutions and material presets for the Margolus implementation

The Margolus implementation shows similar scaling behavior, with very comparable runtimes.

Overall, the benchmark results confirm the expected hierarchy between the sequential and concurrent versions: the sequential CPU version is the least scalable, threading improves performance for sufficiently large grids, and the GPU implementations provide the best interactive behavior at high resolutions.

	Directional Phases	Margolus
1024×1024	8.48	8.23
2048×2048	21.14	20.85
4096×4096	27.31	29.87

Table 5: Speed up of GPU versions with respect to the sequential version

§8 Remarks and Further work

In this falling sand simulation project, we first implemented a sequential implementation to be familiar with the mechanics of a cellular automaton and falling sand, before implementing a concurrent GPU implementation. This transition from C++ and CPU threads to GDScript and GLSL compute shaders allowed us to significantly improve the capability and performance of our game. There are a few things that we recognise could be incorporated, but did not have time to implement ourselves and leave for further work. These include :

- **Expanding the material palette** : the current GPU game features `Air`, `Sand`, `Water`, `Stone`, and `Vapor`. It would be interesting to introduce more dynamic interactions based on temperature (e.g. `Water` freezing to `Ice`, or evaporating to `Vapor`), as well as introducing more complex materials like fire or plants, as is for example done in [sandspiel](#).

- **Sleep States** : Currently, the compute shaders call the update function on every pixel of the grid every frame, regardless of whether a given neighborhood only contains static materials. Implementing a sleep state for inactive neighborhoods of the grid could perhaps improve performance.

References

- [1] Jonathan Devlin and Micah D. Schuster. *Probabilistic Cellular Automata for Granular Media in Video Games*. 2020. arXiv: [2008.06341](https://arxiv.org/abs/2008.06341) [[nlin.CG](#)]. URL: <https://arxiv.org/abs/2008.06341>.